DELHI TECHNICAL CAMPUS

Affiliated to Guru Gobind Singh Indraprastha University, New Delhi

PROJECT BASED LEARNING REPORT

Projectile Motion Calculator

An Interactive Web Application for Trajectory Analysis

Submitted in partial fulfillment of the requirements for

Computational Methods Lab ES 251

Submitted By

Student Name	Enrollment Number
Bhumika	00818015625
Ronit Sharma	01018015625

Section G

Under the Guidance of

Dr. Padmesh Tripathi

Department of Computer Science and Engineering

Batch 2024-28

DELHI TECHNICAL CAMPUS

GREATER NOIDA

Affiliated to GGSIPU and Approved by AICTE & COA

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

PROJECTILE MOTION CALCULATOR

Certificate

This is to certify that the project report entitled "Projectile Motion Calculator" is a genuine record of the work carried out by Bhumika (Enrollment No. 00818015625) and Ronit Sharma (Enrollment No. 01018015625), students of Section G, Department of Computer Science and Engineering, Delhi Technical Campus, under my guidance and supervision.

The project fulfills the academic requirements of Project Based Learning for Computational Methods Lab ES 251. The work presented in this report has been examined and is suitable for submission.

_____	_____
Dr. Padmesh Tripathi	Head of Department
Project Guide	Computer Science and Engineering

Date: _____

PROJECTILE MOTION CALCULATOR

Declaration

We, Bhumika and Ronit Sharma, hereby declare that the project report entitled "Projectile Motion Calculator" has been prepared by us under the guidance of Dr. Padmesh Tripathi. The project work is original and has not been submitted, in full or in part, for the award of any other academic qualification.

All sources of information, software libraries and technical references used during the project have been acknowledged in the references section. We accept responsibility for the accuracy of the work and the results presented in this report.

Name	Enrollment Number	Signature
Bhumika	00818015625	_____
Ronit Sharma	01018015625	_____

Section G

PROJECTILE MOTION CALCULATOR

Acknowledgement

We express our sincere gratitude to Dr. Padmesh Tripathi for his guidance, constructive suggestions and continuous encouragement throughout this project. His feedback helped us connect the numerical methods used in the application with clear visual and interactive results.

We also thank the Department of Computer Science and Engineering, Delhi Technical Campus, for providing the academic environment and resources required to complete the project. We are grateful to our classmates for testing the application on different devices and sharing usability feedback.

Finally, we acknowledge the open-source communities behind React, Vite, Chart.js and Lucide React. Their documentation and software made it possible to build, test and deploy the application efficiently.

Bhumika and Ronit Sharma

PROJECTILE MOTION CALCULATOR

Abstract

The Projectile Motion Calculator is a responsive web application developed to calculate and visualize the motion of a projectile under gravity. The application accepts initial velocity, launch angle, initial height and gravitational acceleration as inputs. It reports maximum height, total flight time, horizontal range and impact velocity while plotting the complete trajectory in real time.

The ideal model uses the standard equations of two-dimensional motion. An optional numerical model introduces quadratic air resistance through small time-step integration. The application also supports metric and imperial units, planetary gravity presets, animated playback, velocity vectors, trajectory comparison, target challenges and energy calculations. Results can be exported as PNG images or CSV data.

The user interface was implemented with React and Vite. Chart.js renders the trajectory graph, while custom JavaScript performs the physical calculations. The project was tested using analytical cases and interaction checks. For the default case of 20 m/s at 45 degrees from ground level under Earth gravity, the application produces values close to the theoretical maximum height of 10.19 m and horizontal range of 40.77 m. The project demonstrates how numerical computation and visualization can make a familiar physics problem easier to understand and explore.

Keywords: projectile motion, numerical simulation, React, Chart.js, trajectory visualization, quadratic drag

PROJECTILE MOTION CALCULATOR

Table of Contents

Section	Page
Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
Chapter 1 Introduction	7
Chapter 2 Literature Review	8
Chapter 3 System Analysis and Requirements	9
Chapter 4 Design and Methodology	10
Chapter 5 Implementation	12
Chapter 6 Testing and Results	14
Chapter 7 Conclusion and Future Scope	15
References	16
Appendix	17

Chapter 1 Introduction

1.1 Background

Projectile motion describes the path of an object launched into the air when gravity is the dominant force. It is a standard example of two-dimensional motion because horizontal and vertical movement can be analyzed separately. Although the governing equations are compact, students often find it difficult to connect changing input values with the complete shape and timing of the trajectory.

1.2 Problem Statement

Manual calculations generally provide one result at a time and do not show how the projectile moves. Static graphs also make comparison difficult when velocity, angle, launch height, gravity or drag changes. A useful learning tool should calculate the results immediately, plot the full path and allow users to explore several physical conditions without rewriting equations.

1.3 Need and Significance

- A visual trajectory makes the relationship between equations and motion easier to interpret.
- Real-time updates support experimentation with launch parameters.
- Planetary gravity and unit conversion expand the range of classroom examples.
- Animation and comparison tools support demonstrations, assignments and self-study.

1.4 Objectives

1. Calculate maximum height, flight time, horizontal range and impact velocity.
2. Plot the trajectory and mark the highest point.
3. Support optional quadratic air resistance through numerical integration.
4. Provide an accessible responsive interface for desktop and mobile screens.
5. Add comparison, target, energy, export and educational features.

1.5 Scope

The project focuses on two-dimensional projectile motion. It supports constant gravity and a simplified quadratic drag model. Wind, spin, lift, changing air density and three-dimensional effects are outside the current scope.

Chapter 2 Literature Review

2.1 Ideal Projectile Motion

In the ideal model, air resistance is neglected and gravitational acceleration remains constant. The horizontal acceleration is zero, while the vertical acceleration equals minus g. If the projectile is launched with speed v_0 at angle θ from initial height y_0, the position at time t is given by:

$$x(t) = v_0 \cos(\theta)t$$

$$y(t) = y_0 + v_0 \sin(\theta)t - \tfrac{1}{2}gt^2$$

The velocity components are:

$$v_x(t) = v_0 \cos(\theta) \quad \text{and} \quad v_y(t) = v_0 \sin(\theta) - gt$$

2.2 Analytical Results

For a launch from ground level that lands at the same height, the maximum height H, flight time T and range R can be written as:

$$H = v_0^2 \sin^2(\theta)/(2g) \qquad T = 2v_0 \sin(\theta)/g \qquad R = v_0^2 \sin(2\theta)/g$$

2.3 Air Resistance

A drag force opposes the direction of motion and depends on speed. The application uses a simplified quadratic model in which acceleration is proportional to speed multiplied by each velocity component:

$$a_x = -k|v|v_x \quad \text{and} \quad a_y = -g - k|v|v_y$$

These coupled equations do not have the same simple closed form as the ideal model, so the application advances the state numerically using a small time step.

2.4 Existing Digital Tools

Many online calculators compute ideal range and height. The present project extends the typical calculator by combining live plotting, playback, drag, trajectory overlays, target placement, energy estimates and export functions in one interface.

Chapter 3 System Analysis and Requirements

3.1 Feasibility Study

Feasibility Area	Assessment
Technical	React handles state-driven updates, Chart.js supports interactive plotting and JavaScript performs the numerical calculation without a backend.
Operational	Users enter familiar physical parameters and receive immediate results. The interface works in a standard web browser.
Economic	The application uses open-source libraries and can be hosted on a low-cost static deployment platform.

3.2 Functional Requirements

- Accept velocity, angle, height, gravity and drag settings.
- Calculate and display four primary motion results.
- Render a trajectory chart with tooltips and apex marker.
- Animate a projectile along the calculated path.
- Save comparison paths and place a target on the graph.
- Export trajectory data and chart images.

3.3 Non-functional Requirements

- Responsive layout for common screen sizes.
- Immediate feedback after input changes.
- Readable units and validated numeric entry.
- Modular source code that separates controls, physics and visualization.

3.4 Hardware and Software Requirements

Category	Requirement
Hardware	Dual-core processor, 4 GB RAM, display and input device
Client software	Modern Chrome, Edge, Firefox or Safari browser
Development tools	Node.js, npm, Vite and Git
Libraries	React, Chart.js, react-chartjs-2 and Lucide React
Deployment	GitHub repository and Vercel production hosting

Chapter 4 Design and Methodology

4.1 System Architecture

The application follows a component-based client architecture. User actions update React state. The physics module receives normalized metric inputs and returns a trajectory object containing coordinate samples, apex data and summary results. Presentation components transform those results into cards, charts, animation frames and export files.

Stage	Responsibility
Input	Capture velocity, angle, height, gravity, units and drag settings.
Normalization	Convert imperial values to SI units and validate limits.
Physics engine	Integrate position and velocity until the projectile reaches the ground.
Result model	Store points, apex, flight time, range and impact speed.
Presentation	Render results, trajectory, playback, comparisons and laboratory tools.

4.2 Calculation Algorithm

6. Convert the launch angle from degrees to radians.
7. Resolve the initial velocity into horizontal and vertical components.
8. Select a stable time step from the estimated flight duration.
9. Apply gravity and optional drag acceleration.
10. Update velocity, position and elapsed time.
11. Record sampled coordinates and update the apex when height increases.
12. When the height crosses zero, interpolate the final step to obtain the impact point.
13. Return the trajectory and derived results to the interface.

Chapter 4 Design and Methodology Continued

4.3 Numerical Integration

The drag model uses a fixed small step selected adaptively from the estimated flight duration. For each step, the program computes acceleration from the current velocity, updates the velocity and then advances the position. The process stops at ground impact or at a protective maximum iteration count. Linear interpolation between the final two vertical positions improves the estimated landing time and range.

4.4 User Interface Design

The interface uses a two-column desktop layout. The left panel contains launch controls and the right workspace contains result cards and the trajectory chart. On smaller screens the layout becomes a single column. Extended functions appear in a laboratory tools dialog so the primary calculation remains easy to use.

4.5 Data Flow

Input	Processing	Output
Velocity and angle	Resolve v_x and v_y	Trajectory coordinates
Initial height and gravity	Integrate vertical motion	Flight time and maximum height
Drag coefficient	Apply speed-dependent acceleration	Reduced range and energy loss
Unit selection	Convert to and from SI units	Metric or imperial display

4.6 Validation Strategy

Ideal-model results are compared with closed-form equations for cases where the launch and landing heights are equal. Interaction tests verify unit conversion, reset behavior, custom gravity entry and responsive layout. Drag results are checked for physically reasonable behavior: range and mechanical energy should decrease as drag increases.

PROJECTILE MOTION CALCULATOR

Chapter 5 Implementation

5.1 Technology Stack

Technology	Use in the Project
React	Component structure, application state and live updates
Vite	Development server and optimized production build
Chart.js	Trajectory plot, tooltips, markers and overlays
Lucide React	Consistent lightweight interface icons
CSS	Responsive layout, themes and interaction styling
Vercel	Production deployment from the GitHub project

5.2 Source Modules

Module	Purpose
App.jsx	Coordinates application state and top-level layout.
Controls.jsx	Implements validated parameter and unit inputs.
physics.js	Calculates ideal and drag trajectories.
Results.jsx	Displays the four main calculated quantities.
TrajectoryChart.jsx	Renders the graph, animation, vector and target.
LabTools.jsx	Provides presets, comparison, energy, export and learning tools.

5.3 Main Features

- Metric and imperial systems with physical-value conversion.
- Earth, Moon, Mars and custom gravity values.
- Optional quadratic air resistance.
- Animated playback with a draggable timeline.
- Trajectory comparison and target challenge.
- Energy dashboard, formulas, insights, exports and display themes.

PROJECTILE MOTION CALCULATOR

Chapter 5 Running Module

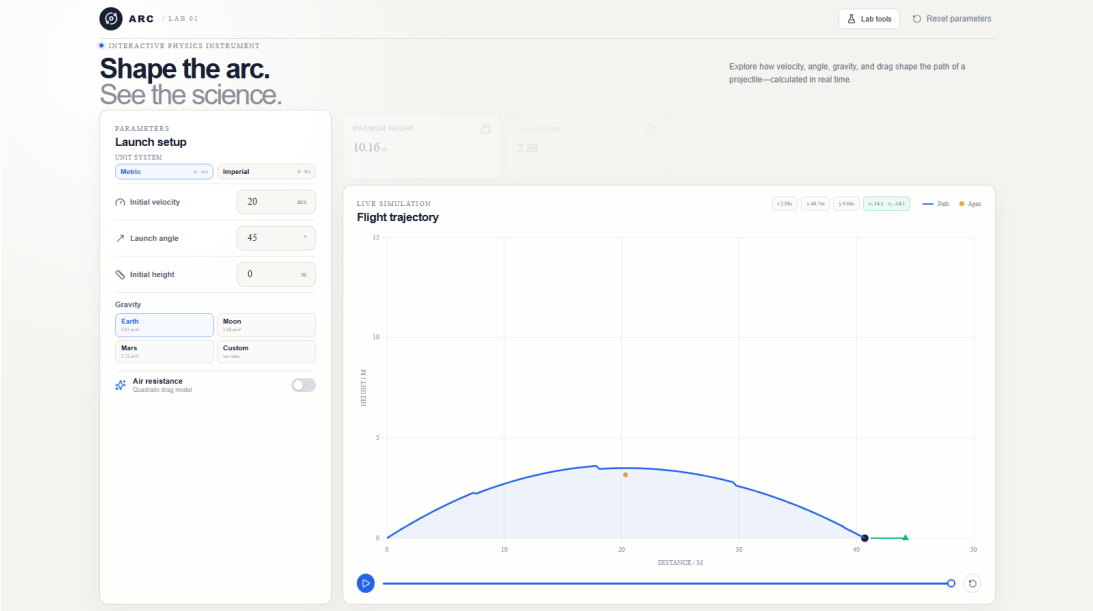


Figure 5.1 Main projectile motion dashboard

The controls panel contains unit selection, launch parameters, gravity presets and drag. The workspace shows calculated results and the live trajectory. Playback controls under the graph animate the current projectile position. Additional experiments are available through the Lab tools button.

5.4 Deployment

The complete source code is stored in a GitHub repository. Vercel builds the Vite project and publishes the optimized dist directory. The deployed application is available at https://projectile-motion-calculator-two.vercel.app.

PROJECTILE MOTION CALCULATOR

Chapter 6 Testing and Results

6.1 Analytical Test Cases

Test	Input	Expected Result	Application Result	Status
Earth default	20 m/s, 45°, $y_0 = 0$	$H \approx 10.19$ m, $R \approx 40.77$ m	$H \approx 10.16$ m, $R \approx 40.7$ m	Pass
Horizontal launch	20 m/s, 0°, $y_0 = 10$ m	Positive flight time and range	Trajectory falls from 10 m	Pass
Vertical launch	20 m/s, 90°, $y_0 = 0$	Range approximately zero	Near-zero horizontal range	Pass
Moon gravity	20 m/s, 45°, $g = 1.62$	Longer time and range than Earth	Longer plotted trajectory	Pass
Air resistance	Drag enabled	Range below ideal case	Range decreases	Pass

6.2 Interface Testing

Area	Check	Result
Numeric input	Clear, replace and enter decimal values	Valid
Custom gravity	Edit without forced spinner or premature clamping	Valid
Reset	Restore metric defaults and clear advanced state	Valid
Responsive layout	Fit desktop frame and stack on mobile	Valid
Exports	Create PNG and CSV outputs	Valid
Production build	Vite build completes successfully	Valid

6.3 Discussion

The default ideal case closely matches analytical values. Small differences arise from the numerical time step used for a consistent trajectory model. The final step interpolation limits the impact error. Drag tests show the expected reduction in range and impact energy, while planetary presets demonstrate the strong effect of gravitational acceleration on flight time and distance.

Chapter 7 Conclusion and Future Scope

7.1 Conclusion

The Projectile Motion Calculator meets its primary objective of combining physical calculation with an immediate visual explanation. It calculates the main motion quantities, plots the trajectory, animates the flight and provides advanced experiments without requiring a backend service. The modular React implementation keeps the calculation logic separate from the interface and makes future extension practical.

Testing confirms that the ideal model agrees closely with standard analytical equations. The custom drag model produces physically reasonable changes in range and energy. The responsive design and deployed web version make the project suitable for classroom demonstration and independent study.

7.2 Limitations

- The drag coefficient represents a simplified combined parameter rather than a full aerodynamic model.
- Gravity and air density remain constant during each simulation.
- Wind, spin, lift and three-dimensional effects are not included.
- The application does not currently store user experiments between sessions.

7.3 Future Scope

- Add wind speed, launch direction and three-dimensional trajectories.
- Model air density changes with altitude and expose mass, area and drag coefficient separately.
- Provide classroom assignments, saved experiments and scored target challenges.
- Add camera-based comparison with a recorded real projectile.
- Package the application as an offline progressive web application.

References

[1] Halliday, D., Resnick, R., and Walker, J. Fundamentals of Physics. Wiley.

[2] OpenStax. University Physics Volume 1. Motion in Two and Three Dimensions. Rice University.

[3] React Documentation. https://react.dev

[4] Chart.js Documentation. https://www.chartjs.org/docs

[5] Vite Documentation. https://vite.dev

[6] Lucide Documentation. https://lucide.dev

[7] GitHub Repository. https://github.com/Bhumika-gaba23/projectile-motion-calculator

[8] Production Application. https://projectile-motion-calculator-two.vercel.app

PROJECTILE MOTION CALCULATOR

Appendix

A Default Project Parameters

Parameter	Default Value
Initial velocity	20 m/s
Launch angle	45 degrees
Initial height	0 m
Gravity	9.81 m/s²
Air resistance	Disabled
Projectile mass	1 kg

B Important Links

Source code: https://github.com/Bhumika-gaba23/projectile-motion-calculator

Live application: https://projectile-motion-calculator-two.vercel.app

C Team Details

Student	Enrollment Number	Section
Bhumika	00818015625	G
Ronit Sharma	01018015625	G

D Project Guide

Dr. Padmesh Tripathi, Computational Methods Lab, Delhi Technical Campus.